

Laborator 2 CNI

Celule de memorare cu circuite logice secvențiale. Regiștri cu acces paralel. Regiștri cu acces serial. Latch-ul adresabil

Conținut laborator:

- 1. Obiectivul lucrării.**
- 2. Noțiuni teoretice:**
 - a. Intrări active și intrări pasive. Circuite logice secvențiale (recapitulare);**
 - b. Regiștri cu acces paralel. Regiștri cu acces serial. Registrul serie – paralel;**
 - c. Latch-ul adresabil.**
- 3. Cerințe laborator.**
- 4. Temă.**

1. Obiectivul lucrării: Laboratorul este dedicat studiului structurilor elementare de memorare, al regiștrilor paraleli și cu deplasare serială (cozi hardware), mecanismul de adresare într-o arie de celule de memorare precum și modul de operare cu structurile de memorare. Prima parte va conține prezentarea unor structuri elementare secvențiale cu descrierea lor în limbajul VHDL. Ulterior, folosind aceste module, se va prezenta construirea unor circuite de memorare extinse și a unor arii de memorare adresabile. Tot aici se vor explica pașii care trebuie parcurși atunci când se lucrează cu un modul de memorare.

Utilitatea laboratorului: Laboratorul prezintă metode de proiectare a unor circuite de memorare, metode care pot fi utilizate ulterior pentru proiectarea unor sisteme de calcul mai complexe. Regiștrii cu acces paralel și serial sunt prezenți în interiorul microprocesoarelor, reprezentând cea mai rapidă memorie care există într-un sistem. Latch-ul adresabil prezintă, într-un mod didactic, mecanismul de adresare care există și în memoriile întâlnite în practică, utilizat pentru accesarea mai multor locații de memorie. Metode eficiente de descriere, utilizând limbajul VHDL, a memoriilor de capacități mari de tip SRAM se vor prezenta în laboratoarele următoare.

2. Noțiuni teoretice:

a. Intrări active și intrări pasive. Circuite logice secvențiale.

Există două tipuri de celule elementare de memorie: cu acces prin nivel (latch-uri) și cu acces prin tranziție (bistabili). Prin tranziție (front) se înțelege comutarea dintr-o stare logică în alta, deci există două tipuri de tranziții: crescătoare și descrescătoare. Nivelul poate fi '0' logic și '1' logic.

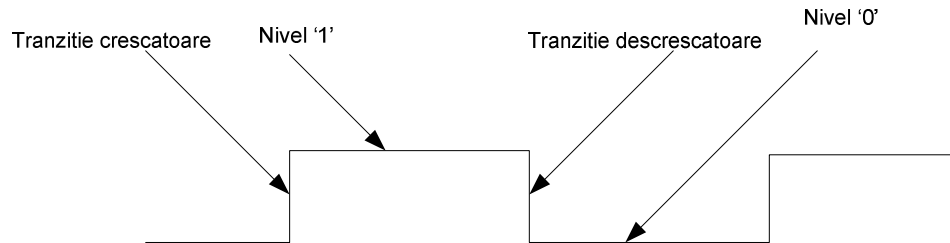


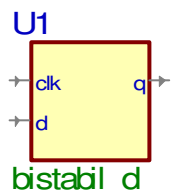
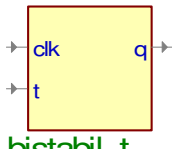
Fig.1. Nivelul și tranziția la semnalul digital

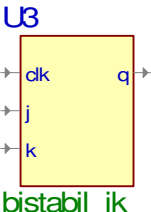
Circuitele care reacționează la fronturi au practic două tipuri de intrări: intrări pasive, intrările care reacționează la nivel, și intrări active, intrări care acționează la un front. De regulă există o singură intrare activă, denumită intrare de ceas sau de sincronizare, și mai multe intrări pasive.

Deși ca arhitectură, circuitele care au numai intrări pasive (circuite combinaționale) au o structură constructivă mai simplă, sunt mai sensibile la stările intermediare. Folosirea intrărilor active (circuite secvențiale), mărește complexitatea dar micșorează sensibilitatea la stările intermediare. Astfel, pe un front, denumit front activ, sunt validate ieșirile la circuit.

Majoritatea circuitelor aflate în calculatoare se bazează pe arhitecturi secvențiale. În această categorie intră inclusiv circuitele de memorie. Cea mai simplă memorie secvențială, denumită celulă elementară de memorie, este bistabilul. În tabelul de mai jos sunt prezentate câteva tipuri de bistabili, modul în care aceștia funcționează și descrierea lor în VHDL.

Tabelul 1:

Denumire	Simbol	Descriere	Descriere VHDL
Bistabilul D (Flip-Flop D - FFD)		Intrarea clk este o intrare cu acces prin tranziție. Pe frontul activ al semnalului clk ieșirea q copiază starea intrării d. Altfel, ieșirea q păstrează vechea stare memorată.	<pre> process(clk,d) variable q_int : std_logic; begin if clk'event and clk = '1' then q_int := d; end if; q <= q_int; end process; </pre>
Bistabilul T (Flip-Flop T - FFT)		Intrarea clk este o intrare cu acces prin tranziție. Pe frontul activ al semnalului clk ieșirea q își schimbă sensul dacă intrarea t = 1 sau își păstrează vechea stare memorată dacă t = 0.	<pre> process(clk,t) variable q_int : std_logic; begin if clk'event and clk='1'then if t = '1' then q_int := not q_int; end if; end if; q <= q_int; end process; </pre>

Bistabilul JK (Flip-Flop JK)		Intrarea clk este o intrare cu acces prin tranziție. Pe frontul activ al semnalului clk starea ieșirii q se schimbă în funcție de stare intrărilor j și k. Dacă intrarea j este în 1 logic iar intrarea k este în 0 logic atunci ieșirea q devine 1 logic. Dacă intrarea j este 0 iar k este 1 ieșirea q devine 0 logic. Dacă ambele intrări jk sunt 1 ieșirea q își schimbă starea anterioară iar dacă ambele sunt 0 q rămâne în vechea stare memorată.	<pre> process(clk,j,k) variable q_int : std_logic; begin if clk'event and clk='1' then if j='1' then if k = '0' then q_int := '1'; else q_int := not q_int; end if; else if k = '1' then q_int := '0'; end if; end if; end if; q <= q_int; end process; </pre>
---	---	---	---

b. Regiștri cu acces paralel. Regiștri cu acces serial. Registrul serie – paralel

Bistabilii, prezențați mai sus, pot fi utilizați în mod individual pentru memorarea unor date pe un bit. De exemplu, bistabilul D are două regimuri de funcționare. Unul, atunci când apare un front activ pe intrarea clk, în care informația plasată pe intrarea d ajunge pe ieșirea q, și cel de-al doilea regim, atunci când nu apare un front activ pe clk, în care informația se păstrează memorată. Putem spune că la apariția unui front activ pe intrarea clk are loc o operație de scriere a memoriei în timp ce informația memorată poate fi citită oricând pe ieșirea q.

Totuși aceste memorii pot stoca un singur număr pe un bit. Destul de des este necesară stocarea unor numere pe mai mulți biți. Pentru aceasta sunt utilizate circuite compuse din mai mulți bistabili, denumite **regiștri**. De exemplu, în cazul în care se dorește memorarea unui număr pe 4 biți se folosește următorul montaj, prezentat în figura 2.

Circuitul este compus din patru bistabili D care au intrările de ceas clk conectate împreună iar pe intrările de date d se află plasat numărul pe 4 biți care se dorește memorat (d3d2d1d0). Pe ieșirile q se poate citi numărul memorat în registru.

Operația de scriere a acestei memorii se face prin generarea unui front activ pe intrarea comună clk.

Acest registru este denumit „**registru cu acces paralel**” dintr-un motiv ușor de observat în figura 2 și anume posibilitatea înscrierii paralele a datei de intrare în acesta.

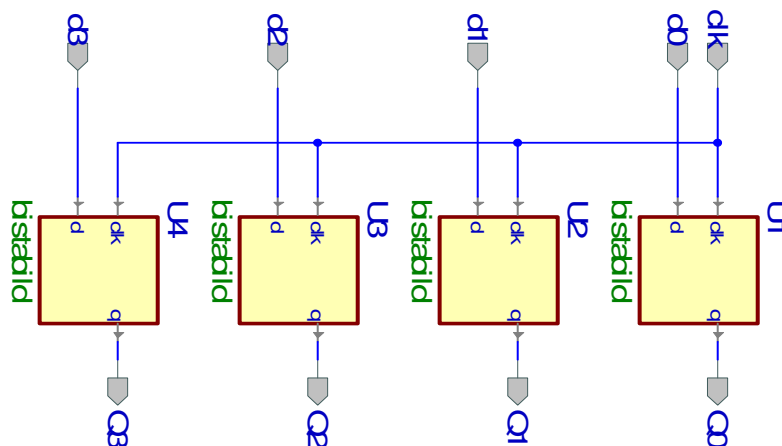


Fig.2. Registru cu acces paralel pe 4 biți (unde nu sunt noduri liniile nu se intersectează!)

Asemenea circuite de memorare pot stoca numere, practic pe oricâți biți, prin extinderea schemei de mai sus. De exemplu, în interiorul microprocesoarelor se află regiștri cu acces paralel, practic cele mai rapide memorii din sistem, deoarece lucrează la frecvențele de lucru ale microprocesorului.

În microprocesor fiecare registru are un nume, desemnat de regulă prin litere sau cifre (exemplu: A, B, C, AX, BX, R0, R1 etc). Accesul la fiecare registru se face separat, prin generarea unui semnal de ceas separat pentru fiecare registru. În figura de mai jos este prezentat un set de regiștri compus din 4 regiștri cu acces paralel: A, B, C, D. Fiecare registru are propriul său semnal de ceas prin care este comandată scrierea în acesta.

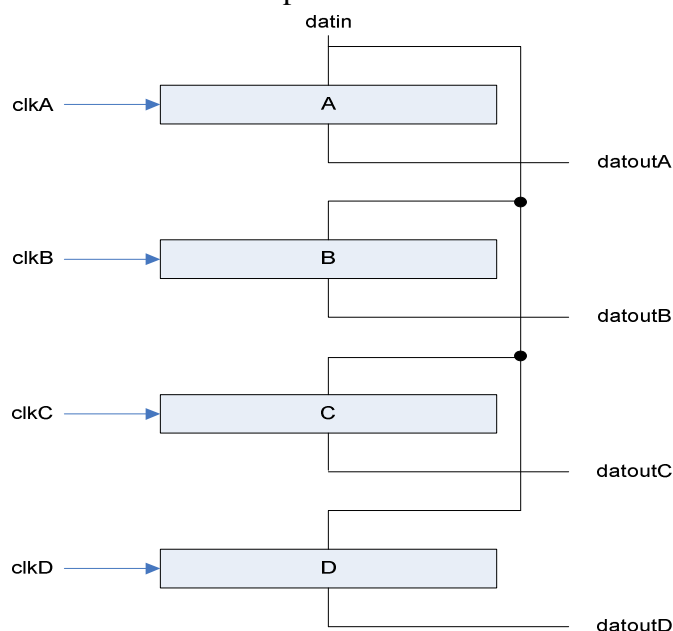


Fig.3. Setul de regiștri

Setul de regiștri poate fi privit ca o memorie care are 4 locații. Accesul pentru scriere la fiecare locație se face prin generarea semnalului de ceas corespunzător. De exemplu pentru a introduce în registrul A o valoare se plasează pe intrările de date valoarea respectivă (datin) apoi se va genera semnalul de ceas corespunzător (clkA).

Accesul pentru scriere este la un singur registru, și aceasta deoarece procesorul nu poate executa decât instrucțiuni secvențiale. În schimb la citire datele sunt toate disponibile, putând fi preluate de alte unități din procesor în același timp.

Un alt set important de circuite de memorare este constituit de regiștri cu acces serial. Aceștia pot memora, ca și regiștrii paraleli, date pe mai mulți biți dar modul în care este înscrisă această informație este unul serial, bit cu bit.

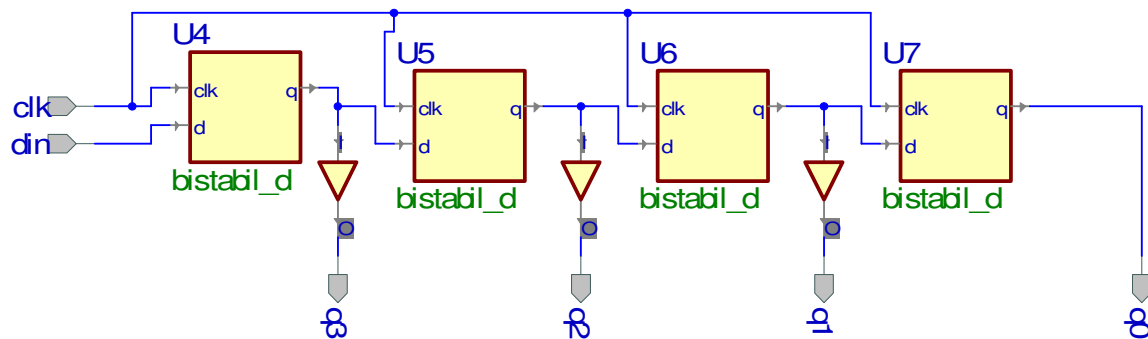


Fig.4. Registrul cu acces serie pe 4 biți

Data de intrare la registrul cu acces serie prezentată în figura 4 se plasează pe intrarea din. Pe intrarea clk este generat un front activ atunci când informația de pe intrare se dorește scrisă în primul bistabil, cel mai din stânga din figura 4. Simultan cu operația de scriere a bistabilului U4 cu valoarea de pe intrarea din figura 4, are loc și transferul informației care se afla pe ieșirea q3 în bistabilul U5, aflat în dreapta, a informației care se găsește pe ieșirea q2 în bistabilul U6 și informația care se găsește pe ieșirea q1 în bistabilul U7. Dacă privim numărul pe patru biți înaintea apariției frontului activ pe intrarea clk și numărul după apariția frontului activ se constată că s-a efectuat o operație de deplasare la dreapta a numărului, cu stocarea pe prima poziție cea mai semnificativă a valorii aflate pe intrarea „din”, așa cum este prezentat în figura 5.

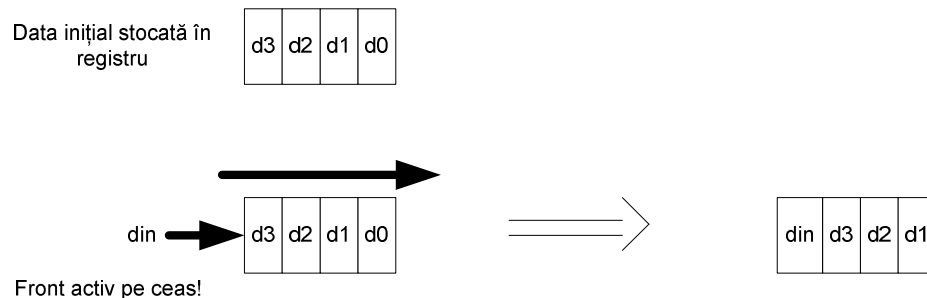


Fig.5. Operația de scriere într-un registru cu acces serial – deplasarea datei anterioare cu actualizarea poziției rămase „liberă”

Operația de deplasare a datelor din interiorul registrului cu acces serie justifică denumirea de „coadă hardware”. Într-adevăr, acesta poate fi privit ca o listă ordonată de biți care urmărește principiul cozii (FIFO) implementată hardware, prin bistabili D. Registrul cu acces serie este utilizat în microprocesoare atât pentru stocarea serială a datelor cât și pentru efectuarea unor operații. Deplasarea la dreapta cu o poziție (un bit)

echivalează cu operația aritmetică de împărțire la 2 iar deplasarea la stânga cu o operație aritmetică de înmulțire cu 2.

Pornind de la structura unui registru cu deplasare la dreapta, prezentat în figura 4 se poate imagina cu ușurință o schemă de registru serie cu deplasare la stânga. De asemenea, schema din figura 4 se poate extinde cu ușurință pentru a concepe un registru cu acces serie pe mai mult de 4 biți.

Destul de des întâlniți în interiorul sistemelor de calcul sunt regiștrii cu acces serie – paralel. Aceștia prezintă caracteristicile registrului cu acces serie și ale celui cu acces paralel. Alături de intrarea de ceas, intrarea de date paralelă și intrarea de date serie aceștia mai prezintă o intrare de selecție a modului în care vor lucra. În modul serie funcționează exact ca un registru cu acces serie. Posedă o intrare serială de date iar la apariția unui front activ pe intrarea clk are loc o deplasare a informației în interiorul acestuia, cu supra scrierea bitului „liber”. În modul paralel, datele plasate pe intrarea paralelă vor fi memorate simultan în toate celulele de memorare. În figura de mai jos (figura 6) este prezentat un registru serie – paralel cu deplasare la dreapta.

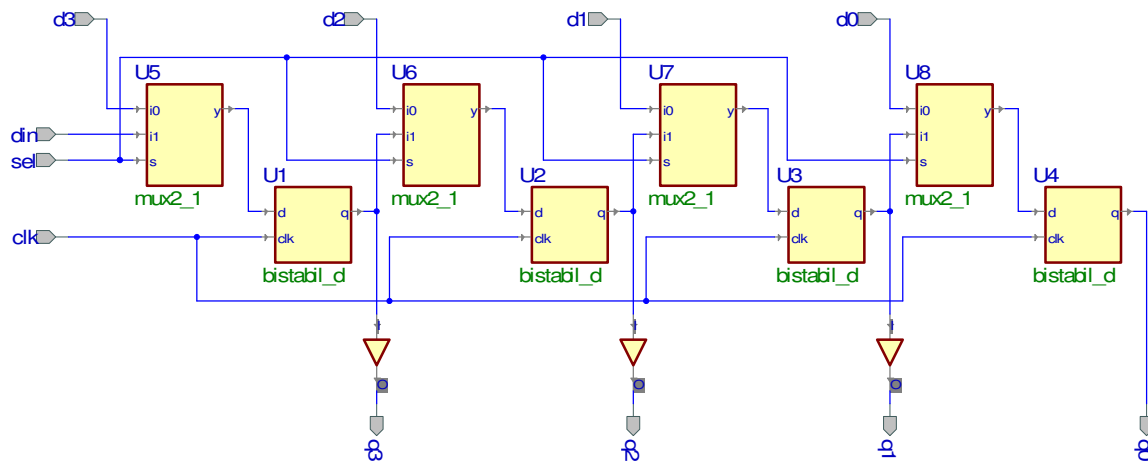


Fig.6. Registrul serie – paralel pe 4 biți cu deplasare la dreapta (unde nu sunt noduri liniile nu se intersectează!)

Intrarea sel din figura 6 joacă rol de intrare de selecție pentru modul de lucru cu registrul serie paralel. Dacă valoarea semnalului pe intrarea sel este 0 logic atunci circuitul se va comporta ca registrul cu acces paralel pe 4 biți, prezentat în figura 2. Pentru a înțelege de ce este suficient să vedem, în figura 6, ce se întâmplă cu multiplexoarele 2:1, notate U5, U6, U7 și U8, atunci când pe intrarea de selecție, comună, a acestora este 0 logic.

Dacă valoarea semnalului pe intrarea sel este 1 logic atunci toate intrările i1 de la multiplexoarele 2:1 vor fi conectate la intrările d ale bistabililor D, din dreapta acestora, așa cum sunt reprezentate în figura 6. Circuitul se va comporta în acest caz ca un registru cu acces serial și cu deplasare la dreapta.

În figura 7 este prezentat un registru serie – paralel cu deplasare în ambele sensuri.

Deși, la prima vedere, schema pare complicată ea poate fi cu ușurință înțeleasă dacă sunt cunoscute modurile de funcționare ale bistabililor D și ale multiplexoarelor 4:1.

Așa cum se observă în figura 7, cele patru multiplexoare 4:1 au intrările de selecție (notate cu s în figură) comune.

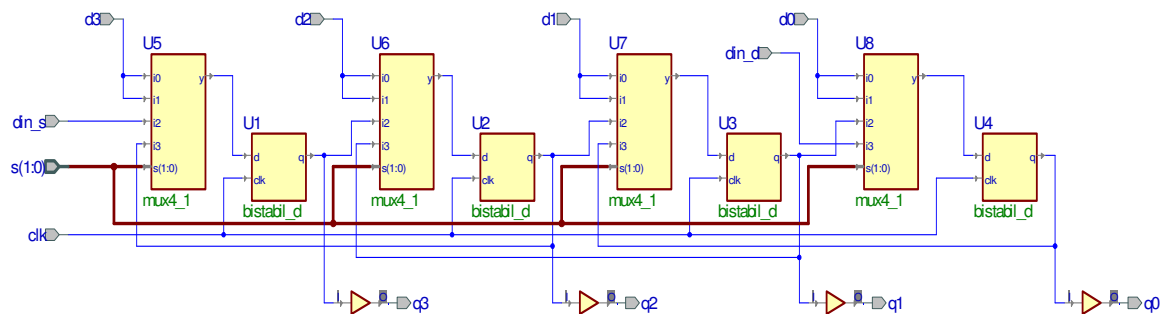


Fig.7. Registrul serie – paralel pe 4 biți cu deplasare la stânga și la dreapta (unde nu sunt noduri liniile nu se intersectează!)

Dacă intrarea cea mai semnificativă de selecție, $s(1)$, este 0 logic atunci pe ieșirea y a multiplexoarelor vor fi conectate intrările $i0$ sau $i1$ în funcție de starea intrării mai puțin semnificative de selecție $s(0)$. Așa cum se poate observa în figură, atât intrarea $i0$ cât și intrarea $i1$ sunt ambele conectate la intrările de date paralele $d3$, $d2$, $d1$ și $d0$ (pentru multiplexorul U5, U6, U7 respectiv U8). Deci, indiferent de starea intrării de selecție $s(0)$, dacă intrarea de selecție $s(1)$ este 0 logic, schema din figura 7 se reduce la schema din figura 2 adică la cea a unui registru cu acces paralel.

Pe de altă parte, dacă intrarea $s(1)$ de selecție este 1 logic, pe ieșirile y ale multiplexoarelor sunt selectate intrările $i2$ sau $i3$ în funcție de starea intrării de selecție $s(0)$. Dacă $s(0)$ este 0 logic atunci pe ieșirea y a fiecărui multiplexor 4:1 ajunge intrarea $i2$. Urmărind schema din figura 7 observăm că în acest caz pe intrările de date d ale bistabililor U2, U3 și U4 ajung ieșirile de la bistabili din stânga lor sau din_s pe intrarea de date d a bistabilului U1. Schema se reduce deci la figura 4, registru cu acces serie cu deplasare la dreapta. În fine, dacă intrarea $s(0)$ este 1 logic atunci pe ieșirile y ale multiplexoarelor 4:1 sunt conectate intrările $i3$ deci pe intrările de date ale bistabililor U1, U2 și U3 ajung ieșirile de la bistabili din dreapta lor iar pe intrarea de date a bistabilului U4 ajunge intrarea din_d . Deci, schema se reduce la un registru cu acces serie cu deplasare la stânga.

Seturile de regiștri ale unui microprocesor pot conține, alături de regiștrii cu acces paralel, și regiștri cu acces serie sau regiștri serie – paralel pentru efectuarea operațiilor de deplasare, utilizate în aritmetica microprocesorului.

c. Latch-ul adresabil

Dimensiunea unei înregistrări care se poate accesa în paralel reprezintă un parametru al circuitelor de memorare. Alături de acesta un alt parametru este numărul de locații distincte care se pot memora. De exemplu, dacă fiecare din regiștrii din figura 3 au 8 biți atunci setul de regiștri din figura 3 poate fi privit ca o memorie cu organizarea 4×8 adică are 4 locații distincte a câte 8 biți fiecare. Se mai notează și memorie 4B (4 bytes) sau 4o (4 octeți). În cazul în care o locație are 1 bit se notează cu litera b (bit).

În cazul memoriilor folosite în practică numărul de locații diferite care le au este foarte mare. Spre exemplu, o memorie a cărei capacitate este de 128Mb (128 mega biți)

are de fapt $2^7 \cdot 2^{20} = 134\,217\,728$ locații! Deci, folosind o metodă de acces la locație de tipul celei utilizate la setul de regiștri, ar trebui un număr foarte mare de semnale de ceas generate.

Practic nu este posibil de realizat așa ceva și din această cauză se folosesc mecanisme de adresare.

Un asemenea mecanism este prezentat în figura de mai jos:

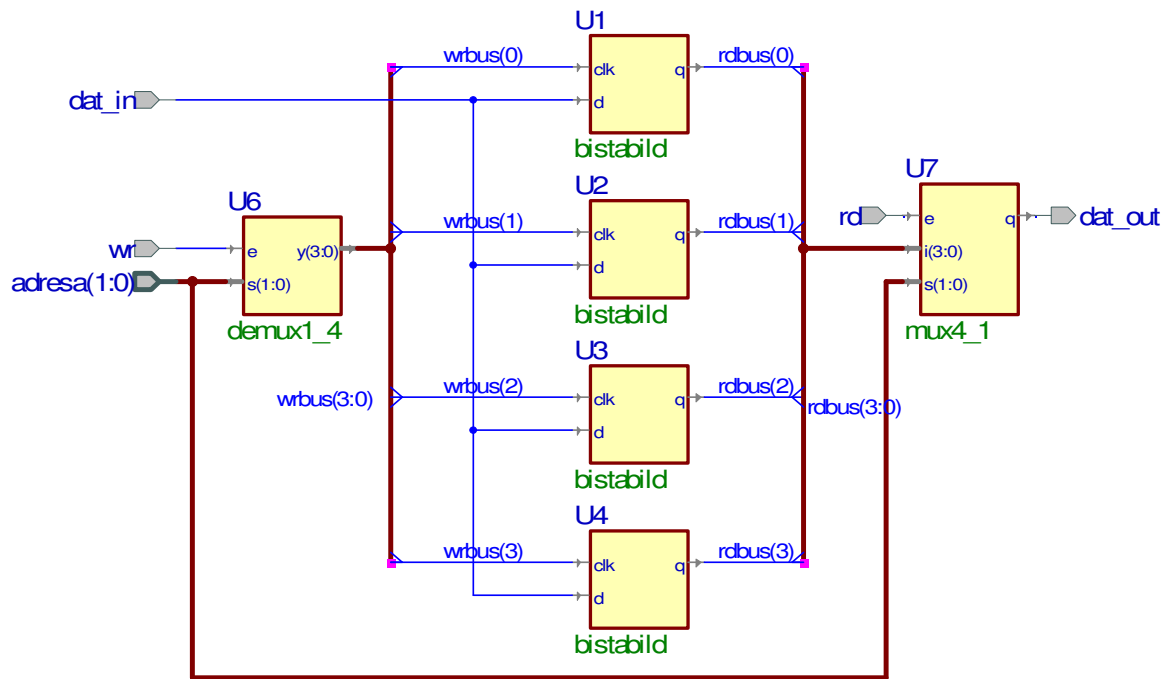


Fig.8. Latch-ul adresabil (memorie 4x1 – 4b)

Ca celule elementare de memorare se folosesc bistabilii D. Semnalele de ceas de data asta sunt generate de la ieșirea unui circuit demultiplexor 1:4 (U6 în figura 8). Astfel, pentru o memorie cu organizarea 4x1, în loc de 4 semnale de ceas distincte, pentru fiecare celulă, vor fi 2 semnale care fac selecția ceasului generat (adresa(1) și adresa(0)). Fiecare ieșire de date din bistabil este multiplexată, prin intermediul unui multiplexor 4:1 cu intrare de activare (U7 în figura 8) obținând o singură ieșire de date pentru toată memoria.

Eficiența schemei de adresare crește logaritmically cu numărul celulelor adresate.

De exemplu, pentru un circuit de memorare de 128Mb, numărul liniilor de adrese este de:

$\log_2(2^{20} \cdot 2^7) = 27$. Prin urmare cu 27 de linii se poate accesa oricare din celula dintr-o memorie de 128Mb.

Tot în figura de mai sus (figura 8) se observă apariția unor semnale suplimentare, necesare pentru lucru cu circuitul astfel proiectat. De exemplu, semnalul notat wr – din eng. **w**rite – (activ 1 logic în figură) este de fapt semnalul care activează demultiplexorul care generează semnalele de ceas pentru fiecare bistabil în parte. Cu alte cuvinte, dacă acest semnal este 0 atunci oricare din ieșirea circuitului demultiplexor va fi 0 deci

bistabilii se vor afla în starea de memorare. Dacă semnalul este 1 atunci una din ieșiri va deveni 1 iar semnalul de ceas, care va intra în bistabil, va tranzita din 0 în 1. O asemenea tranziție va avea drept efect, așa cum s-a arătat la începutul laboratorului, copierea intrării pasive de date d la ieșirea q a bistabilului deci, o modificare a valorii bistabilului respectiv cu valoarea care se află pe linia „dat_in”. Deci putem spune că a avut loc o scriere în celula respectivă de memorie.

Un alt semnal de comandă al memoriei este semnalul rd – din eng. **read** – (activ 1 logic), ce reprezintă de fapt semnalul de validare al multiplexorului. Așa cum acesta a fost descris, când este 0 logic multiplexorul este inactiv, adică ieșirea sa este în înaltă impedanță. Când rd este 1 logic multiplexorul va avea pe ieșire una din intrări selectată de intrarea s(1..0). Intrarea de selecție a multiplexorului este conectată, așa cum se vede în figură, în comun cu intrarea de selecție a demultiplexorului și constituie intrarea de adresă pentru circuitul de memorie.

Denumirea WR provine de la „write” (scrie) și reprezintă semnalul care va determina scrierea în memorie în timp ce RD de la „read” citirea.

Semnalele „adresa”, „rd”, „wr”, „dat_in” și „dat_out” sunt semnale tipice pentru orice circuit de memorare. Rezumând cele spuse mai sus, în lucru cu memoria prezentată în figura de mai sus se efectuează următorii pași pentru operația de scriere și citire:

Scriere

- 1. Se introduce adresa locației la care se dorește să se efectueze scrierea și data de intrare care se dorește scrisă.**

Ex: adresa $\leftarrow$ “00”
dat_in $\leftarrow$ 1

- 2. Se activează semnalul de scriere wr.**

Ex: wr $\leftarrow$ 1

- 3. Se dezactivează semnalul de scriere wr.**

Ex: wr $\leftarrow$ 0.

Citare

- 1. Se introduce adresa locației la care se dorește să se efectueze citirea.**

Ex: adresa $\leftarrow$ “00”

- 2. Se activează semnalul de citire RD.**

Ex: rd $\leftarrow$ 1

- 3. Se dezactivează semnalul de citire RD.**

Ex: rd $\leftarrow$ 0.

Toate comenzile enumerate mai sus pornesc de la idea că inițial circuitul are toate operațiile dezactivate, adică $WR \leftarrow 1$, $RD \leftarrow 1$.

În schema de mai jos sunt prezentate rezultatele simulării pentru următoarele operații: Scrie adresa 0 data 1, citește adresa 0, scrie adresa 1 data 0, citește adresa 0, citește adresa 1.

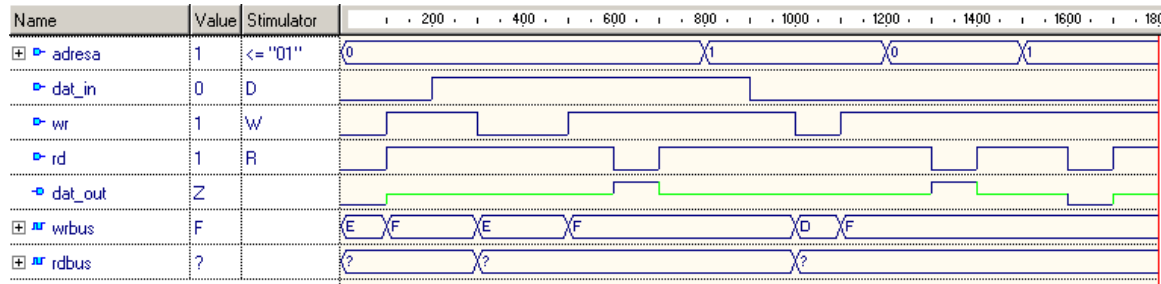


Fig.9. Formele de unda la operațiile de scriere și citire din memoria 4x1

Pentru circuitele de memorare prezente în practică, fiecare din etapele prezentate la operațiile de scriere sau citire au un anumit interval de timp în care trebuie menținute. Aceste intervale de timp sunt date valoric în foile de catalog ale memoriilor respective, și sunt prezentate detaliat la curs.

Condițiile impuse de acest mod de adresare sunt următoarele:

- la un moment dat o singură locație este disponibilă pentru scriere;
- la un moment dat o singură locație este disponibilă pentru citire.

Aceste condiții, dat fiind caracterul secvențial al structurilor de calcul existente la ora actuală, nu reduc din performanțele sistemului. Astfel, la un moment dat, oricum un procesor nu poate executa decât o singură instrucțiune de preluare a datelor din memorie și una singură de scriere.

Totuși s-au construit arhitecturi de procesoare capabile să preia în același timp două locații deodată. În aceste situații se lucrează cu circuite de memorie împerecheate care furnizează, pe aceeași magistrală, două locații în același timp.

3. Cerințe laborator:

1. Să se creeze un proiect în Active HDL și să se descrie, utilizând limbajul VHDL, următoarele circuite logice: multiplexor 2:1, multiplexor 4:1, multiplexor 4:1 cu validarea ieșirii, demultiplexor 1:4, bistabil D.

Rezolvare

Pentru descrierea circuitelor se va construi un proiect, respectând etapele prezentate în laboratorul anterior. Pentru fiecare circuit în parte se va construi un fișier VHDL, folosind instrumentul Wizard din ActiveHDL, așa cum s-a arătat în laboratorul anterior și în anexa. Denumirile fișierelor create vor fi: pentru multiplexorul 2:1 denumirea fișierului vhd care va conține descrierea lui va fi mux2_1.vhd, pentru multiplexorul 4:1 este mux4_1.vhd, pentru multiplexorul 4:1 cu validarea ieșirii este mux4_1_e, pentru demultiplexorul 1:4 este dmux1_4.vhd iar pentru bistabilul D este bistabil.vhd.

Pentru detalii legate de construirea fișierelor pentru descrierea în VHDL consultați laboratorul 1 și anexa.

Metodele de descriere pentru multiplexoare și demultiplexoare le regăsiți în laboratorul 1. Pentru descrierea bistabilului D utilizați circuitul și descrierea VHDL prezentate în tabelul 1.

Se vor compila fișierele care conțin descrierile circuitelor.

2. Să se construiască un registru cu acces paralel pe 4 biți utilizând bistabili D, să se simuleze comportamentul acestuia în active HDL și apoi să se implementeze pe macheta de laborator.

Rezolvare:

Proiectul în care se lucrează trebuie să conțină fișierul bistabild.vhd creat la punctul anterior care conține descrierea în limbajul VHDL a bistabilului D. Fișierul bistabild.vhd trebuie compilat pentru a putea fi utilizat pe post de componentă. Se va construi un fișier de tip schematic (punctul B din anexă, descrierea folosind editorul schematic). În acest fișier se va utiliza ca o componentă descrierea bistabilului D făcută în bistabild.vhd.

Pentru aceasta, din lista cu piese (accesul la aceasta este descris în anexă la subpunctul a în secțiunea dedicată descrierii folosind editorul schematic de la punctul B), se va selecta catalogul cu numele proiectului creat în care se va afla catalogul „Units without symbols” (figura 10).

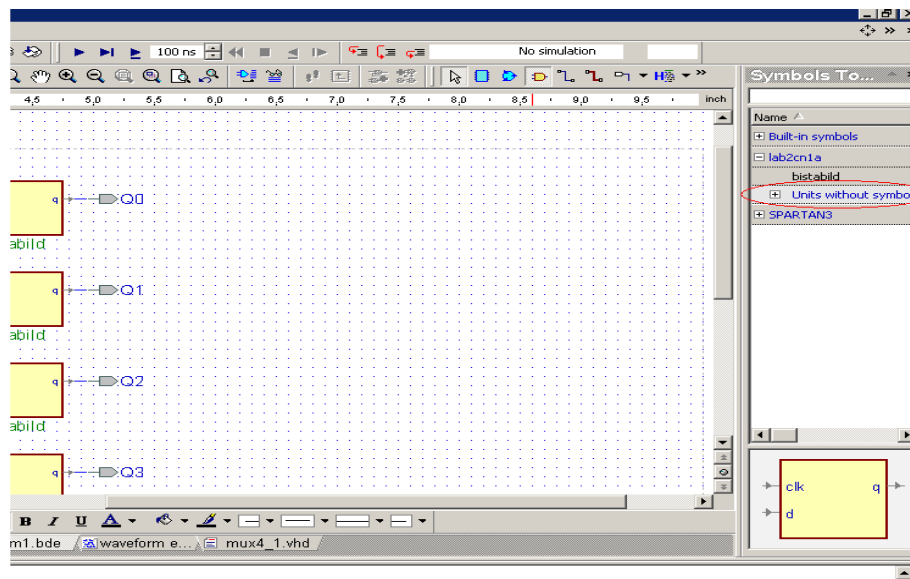


Fig.10. Adăugarea unei componente create într-un proiect

Se va construi astfel registrul cu acces paralel descris în figura 2 din acest laborator.

Se va simula comportamentul acestuia, plasând pe intrările paralele de date d3..d0 stimul de tipul „hotkey” sau „value” în timp ce pe intrarea de ceas clk stimul de tipul „clock”. Despre plasarea stimulilor la simulare și rularea simulării vedeți anexa, punctul D.

Înscrierea unei valori în registrul cu acces paralel se face astfel:

- Se plasează informația care se dorește înscrisă în registru, pe intrările d3..d0;
- Se generează un front pozitiv de ceas pe intrarea clk.

Se va implementa pe machetă, executând etapele de sinteză și implementare descrise în anexă, punctul E, conectând registrul ca în figura și în tabelul de mai jos la pinii machetei.

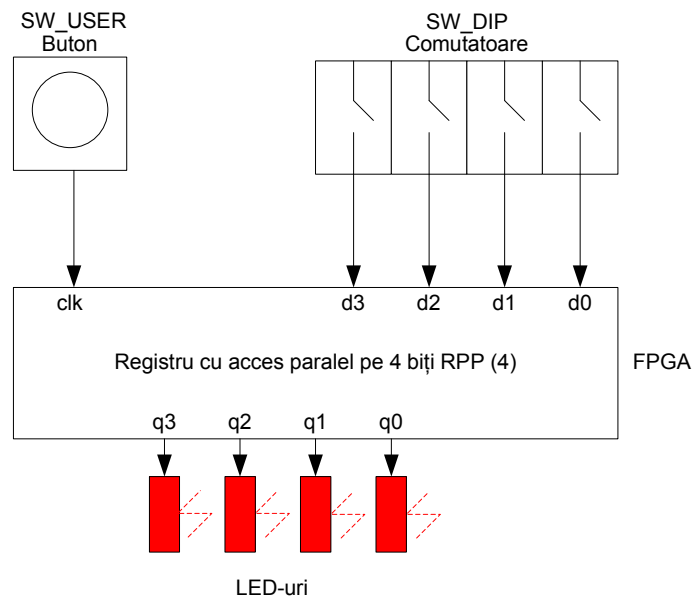


Fig.11. Conectarea registrului cu acces paralel pe 4 biți la macheta cu FPGA

Port logic	Circuit machetă
clk	sw_user0
d3	sw_dip0
d2	sw_dip1
d1	sw_dip2
d0	sw_dip3
q3	led0
q2	led1
q1	led2
q0	led3

3. Să se construiască un registru cu acces serie, deplasare la dreapta. Să se simuleze și să se implementeze pe macheta cu FPGA.

Rezolvare

Procedând similar ca la problema 2 se va construi schema registrului cu acces serie, descrisă în figura 4.

Înscrierea unei valori în registrul cu acces serie se face astfel:

- Se plasează informația care se dorește înscrisă în registru pe intrarea „din”;
- Se generează un front crescător de ceas pe intrarea clk.

Pentru implementare pinii alocați la machetă vor fi următorii:

Port logic	Circuit machetă
clk	sw_user0
din	sw_dip0
q3	led0
q2	led1
q1	led2
q0	led3

4. Să se construiască un registru serie paralel cu deplasare la dreapta pe 4 biți. Să se simuleze comportamentul acestuia și să se implementeze pe macheta din laborator cu FPGA.

Rezolvare

Pentru construirea acestui circuit sunt necesari circuite bistabili D și circuite multiplexoare 2:1. Acestea se vor găsi descrise în fișierele bistabild.vhd și mux2_1.vhd construite la punctul 1, care trebuie să se găsească în proiectul unde se va construi registrul serie-paralel.

Se va construi registrul serie – paralel, prezentat în figura 6.

La simulare, pe intrările paralele de date d3..d0 și pe intrarea serială din se vor asocia stimuli de tipul „value” sau „hotkey” iar pe intrarea de ceas un stimul „clock”.

Scierea unei valori paralel în registru se face în următorii pași:

- Se plasează informația paralelă care se dorește înscrisă în registru pe intrările d3..d0;
- Se configurează intrarea sel în 0 logic, pentru ca registru să opereze în modul paralel;
- Se generează un front pozitiv de ceas pe intrarea clk.

Scierea unei valori serial în registru se face în următorii pași:

- Se plasează informația (1 bit) care se dorește inscripționată serial în registru pe intrarea „din”;
- Se configurează intrarea sel în 1 logic, pentru ca registrul să opereze în modul serial;
- Se generează un front pozitiv de ceas pe intrarea clk.

5. Să se construiască o memorie cu organizarea 4x1 (4b) utilizând latch-ul adresabil cu bistabili D. Să se simuleze comportamentul circuitului. Să se implementeze în macheta de laborator. În etapa de simulare și pe macheta de laborator după implementare să se realizeze următoarele operații:

- La adresa 0 scrie valoarea 1;
- La adresa 1 scrie valoarea 0;
- La adresa 2 scrie valoarea 1;
- La adresa 3 scrie valoarea 0;
- Citește de la adresa 0;

- Citește de la adresa 1;
- Citește de la adresa 2;
- Citește de la adresa 3.

Rezolvare

Pentru construirea memoriei cu organizarea 4x1 (4 biți) prezentată în figura 8 sunt necesare următoarele circuite: demultiplexor 1:4, multiplexor 4:1 cu validarea ieșirii și bistabil D.

Pentru aceasta în proiectul în care se va construi registrul serie-paralel vor trebui să fie prezente și compilate fișierele bistabil.d.vhd, demux1_4.vhd și mux4_1_e.vhd.

Se va crea un fișier schematic în care se va construi schema latch-ului adresabil, de la figura 8. Pentru aceasta se vor folosi componentele descrise mai sus (mux4:1 cu validare ieșire, demux1:4 și bistabil).

Pentru liniile de magistrala (pe mai mulți biți) se folosește instrumentul Bus (lângă Wire). Conectarea unui fir sau a unui fragment de magistrală la o magistrala se face astfel:

a. se selectează magistrala, efectuându-se dublu click cu mouse-ul în orice loc pe ea. În fereastra care apare se introduce numele magistralei și dimensiunea ei, ca în figura de mai jos:

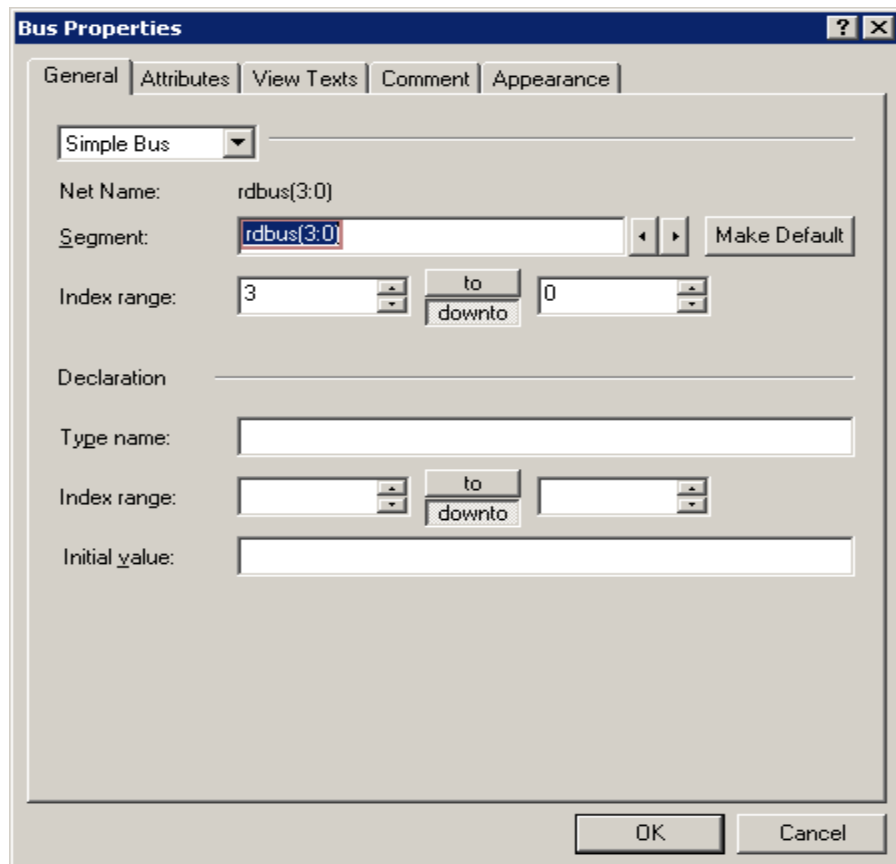


Fig.12. Denumirea unei magistrale și stabilirea dimensiunii acesteia

b. se va selecta firul, sau fragmentul de magistrală, iar în fereastra se va introduce numele magistralei la care se conectează, așa cum a fost denumită la pasul 1, și: între

paranteze numărul bitului extras din magistrala, în cazul conexiuni unui fir, ca în figura de mai jos:

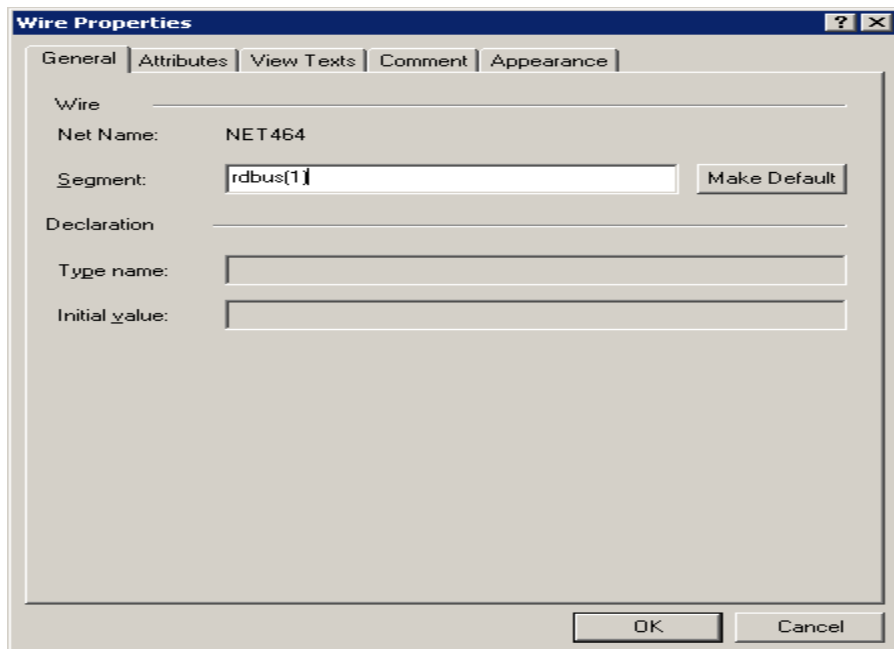


Fig.13. Denumirea unui fir care pornește dintr-o magistrală de date

sau dimensiunea fragmentului de magistrala specificată în câmpul Index Range în cazul conectării unui fragment de magistrala.

La simularea comportamentul latch-ului adresabil, se plasează pe pinii de intrare următorii stimuli:

Intrare	Stimul
Adresa	Value (2 biți)
Dat_in	Tasta D
Wr	Tasta W
Rd	Tasta R

Se va ține cont de faptul ca simulatorul pleacă implicit cu toate intrările în 0!

La implementarea latch-ului adresabil pe macheta cu FPGA Spartan 3 se vor urmări etapele prezentate în anexă. Pinii alocați la machetă vor fi următorii:

Port	Pin macheta
Adresa(0)	Sw_dip1
Adresa(1)	Sw_dip0
Dat_in	Sw_dip2
Rd	Sw_dip3
Wr	Sw_user1

Dat_out	Led0
---------	------

La implementarea pe machetă semnalele care au alte valori decât 0 și 1 (de exemplu Z) vor fi trecute în 0!

Operația de scriere a unei locații din memorii cuprinde următoarele etape, descrise și în tratarea teoretică a circuitului.

Scriere

- Se introduce adresa locației la care se dorește să se efectueze scrierea și data de intrare care se dorește scrisă.
În acest caz, adresa are valori cuprinse între 0 și 3 inclusiv. Există 2 biți de adresă deci valoarea adresei va trebui transformată în formatul binar pe 2 biți. De exemplu 0 = 00b, 1 = 01b, 3 = 11b.
Bitul cel mai puțin semnificativ va fi setat pe adresa(0) iar bitul cel mai semnificativ pe adresa(1).
adresa(1) $\leftarrow$ adresa high
adresa(0) $\leftarrow$ adresa low
- Se activează semnalul de scriere wr.
wr $\leftarrow$ 1
- Se dezactivează semnalul de scriere wr.
wr $\leftarrow$ 0.

Citare

- Se introduce adresa locației la care se dorește să se efectueze citirea.
adresa(1) $\leftarrow$ adresa high
adresa(0) $\leftarrow$ adresa low
- Se activează semnalul de citire rd.
rd $\leftarrow$ 1
- Se dezactivează semnalul de citire rd.
rd $\leftarrow$ 0.

4. Temă

1. În doi regiștri cu acces serie pe 8 biți cu deplasare la dreapta sunt stocate inițial valorile: 80h, 55h. Dacă pe intrarea de date serială (din) la ambii regiștri se află plasată valoarea 0, să se arate care vor fi valorile stocate în aceștia după generarea unui front activ pe intrările clk.
2. În doi regiștri cu acces serie pe 8 biți cu deplasare la stânga sunt stocate inițial valorile: 33h, 40h. Dacă pe intrarea de date serială (din) la ambii regiștri se află plasată valoarea 1, să se arate care vor fi valorile stocate în aceștia după generarea unui front activ pe intrările clk.
3. Să se construiască un registru serie paralel cu deplasare la dreapta pe 8 biți

4. Să se construiască un registru cu acces serie pe 8 biți cu deplasare la stânga
5. Să se realizeze un circuit de memorare cu organizarea 8x1 utilizând modelul latch-ului adresabil. Să se prezinte secvența care asigură inițializarea tuturor locațiilor din această memorie cu valoarea 0. Simulare.
6. Să se proiecteze setul de regiștri pe 8 biți de la microprocesorul Z80. Acesta are următorii regiștri pe 8 biți: A, B, C, D, E, H și L. Toți regiștrii permit încărcarea paralelă a datelor. Registrul A (denumit Acumulator) permite în plus și operații de deplasare la stânga și la dreapta. Simulare
7. Să se proiecteze o memorie cu organizarea 8x4. Să se prezinte secvența care determină stocarea la adresa 5 a valorii 10 și la adresa 7 a valorii 14. Simulare.

Criterii de evaluare:

Laborator	
1	1P
2	2P
3	2P
4	2P
5	2P
Temă	
1	1P
2	1P
3	2P
4	2P
5	2P
6	3P
7	3P